Manually Converting Data Types

The remaining data types you have to convert explicitly by calling the conversion function listed below:

Conversion method	Data type of the return value
bool_to_num [SimTalk](boolean)	*real* [SimTalk]
num_to_bool [SimTalk](integer)	*boolean* [SimTalk]
str_to_bool [SimTalk](string)	*boolean* [SimTalk]
str_to_date [SimTalk](string)	*time* [SimTalk]
str_to_dateTime [SimTalk](string)	*dateTime* [SimTalk]
str_to_length [SimTalk](string)	*length* [SimTalk]
str_to_num [SimTalk](string)	*real* [SimTalk]
str_to_obj [SimTalk](string)	*object* [SimTalk]
str_to_speed [SimTalk](string)	*speed* [SimTalk]
str_to_time [SimTalk](string)	*time* [SimTalk]
str_to_weight [SimTalk](string)	*weight* [SimTalk]
to_str [SimTalk](any, ...)	*string* [SimTalk]

Operators and Expressions

Operators and Expressions

The most basic and most simple form of an expression is a **constant** value or a **variable** value. By combining constants or variables and operators, you can create increasingly complex expressions.

Remarks

The order in which *Plant Simulation* analyzes complex expressions depends on the priority of the operators involved.

Plant Simulation provides **arithmetic operators**, the **logical operators AND and OR**, the **logical operator NOT**, and **relational operators**. Some operators are restricted to certain data types. Logical operators in *Plant Simulation* combine boolean expressions for example.

Arithmetic Operators

SimTalk supports the basic arithmetic operators.

Remarks

Plant Simulation supports the following arithmetic operators:

- **Addition** (+), **subtraction** (-), **multiplication** (*), and **floating-point division** (/) compute a new number from two given numbers.

```
x += y      // adds a value (short for x := x + y)
x -= y      // subtracts a value (short for x := x - y)
x *= y      // multiplies a value (short for x := x *y)
1 + 2  * 3  // returns 7
(1 + 2) * 3 // returns 9
```

  For objects of type *Variable* and for *user-defined attributes* of data types *time*, *length*, *weight*, *speed*, and *acceleration* the operators += and -= check the type more accurately than in previous versions.

  The following source code will throw an error for example:

```
var len : length
VariableObjectOfTypeSpeed += len   // not allowed any longer
len += VariableObjectOfTypeSpeed   // was never allowed
```

- The **integer division** (//), defined for the data type *integer*, returns another *integer* as the result and ignores the remainders.

```
15 div 5 // returns 3
17 div 5 // also returns 3
```

- The **modulo operation** (mod) returns the remainder of an *integer division* which the division of *integer* values suppresses.

```
15 mod 5 // returns 0
17 mod 5 // returns 2
Variable := Variable mod 360
```

  Use the modulo operation to find out if an *integer* is an **odd number** or an **even number**.

  You might, for example, type in:

```
14 mod 2 = 0
15 mod 2 = 1
```

The **modulo operation** (mod) also works for floating point values:

```
print 6.123 mod 2.5 // outputs 1.123
```

See also

SimTalk 2.0 and SimTalk 1.0 Compared

Video on YouTube

https://youtu.be/MXDcx2yplxc?si=IO5x6qj0HkO64qp2&t=396

Relational Operators

The **relational operators** compare two values. The result has the data type *boolean*.

Remarks

Plant Simulation provides the following relational operators:

Operator	Operator Name	Result of the Comparison
=	equal to	**true** if left and right side are equal, otherwise **false** **true** if strings are case-sensitive
/= !=	not equal to	**true** if left and right side are not equal, otherwise **false**
<	less than	**true** if left side is less than right side, otherwise **false**
<=	less than or equal to	**true** if left side is less than right side or equal, otherwise **false**
>	greater than	**true** if left side is greater than right side, otherwise **false**
>=	greater than or equal to	**true** if left side is greater than right side or equal, otherwise **false**
~=	about equal to	**true** if left and right side are about equal, otherwise **false** *
<~=	less than or about equal to	**true** if left side is less than right side or about equal, otherwise **false** *
>~=	greater than or about equal to	**true** if left side is greater than right side or about equal, otherwise **false** *

About Equal Operators

Note

* The **about equal operators** ~=, <~=, and >~= ignore the case when comparing *strings* and ignore value differences less than **epsilon** when comparing numbers. For numerical values you can set the **Tolerance for About Equal Comparison**..

```
print "a" = "A"              -- false
print "a" ~= "A"             -- true
print 1 = 1.00000000000001   -- false
print 1 ~= 1.00000000000001  -- true
```

You can search for a **fuzzy match** with the **about equal operator** ~=:

```
-- fuzzy match 1: ignores upper and lower case
"hello" ~= "HELLO"

-- fuzzy match 2: ignores leading & trailing spaces
strTrim("  hello  ") ~= strTrim("hello    ")

-- fuzzy match 3: ignores both of the above
strTrim(s1) ~= strTrim(s2)
```

Note

Searching for a **fuzzy match** might fail because the *string* contains invisible characters, for example control characters.

Compare Storage Places on Objects and Objects with the about equal operator

All material flow objects, which can receive *MUs*, have one or more storage places.

The about equal operator ~= returns true under these conditions:

- If both values have the data type *object* and if the objects are identical.
- If both values are of data type *storage place*, and if both storage places are located on the same object.
- If one value has the data type *storage place* and the other value has the data type *object* and if the storage place is located on the object.

```
var L : object := "Store"
var L11 : any := Store[1,1]
var L23 : any := Store[2,3]
var W : object := TwoLaneTrack
var WA : any := TwoLaneTrack.A
var WB : any := TwoLaneTrack.B

print L ~= L11    -- true
print L11 ~= L23  -- true
print W ~= WA     -- true
print WA ~= WB    -- true

print L ~= W      -- false
print L11 ~= WA   -- false
```

Relational Operators

The **relational operators** = and /= support comparing the following data types:

Left Side [SimTalk]	Right Side [SimTalk]
integer, real, length, weight, speed, time, acceleration [SimTalk]	integer, real, length, weight, speed, time, acceleration [SimTalk]
date, dateTime [SimTalk]	date, dateTime [SimTalk]
string [SimTalk]	string [SimTalk]
boolean [SimTalk]	boolean [SimTalk]
object [SimTalk]	object [SimTalk]
table, list, stack, queue [SimTalk]	table, list, stack, queue [SimTalk]

Note

When comparing values of data type *object*, *Plant Simulation* always compares the absolute object paths. *Plant Simulation* resolves relative paths before comparing them.

Note

When comparing values of data type *table*, *list*, *stack*, and *queue Plant Simulation* checks if it is the same *list*/*table*. The **relational operator** = returns the value false if a *list*/*table* is compared with a copy.

The **relational operators** <, <=, >=, >, ~=, <~= and >~= support comparing the following data types:

Left Side [SimTalk]	Right Side [SimTalk]
integer, real, length, weight, speed, time, acceleration [SimTalk]	integer, real, length, weight, speed, time, acceleration [SimTalk]
date, dateTime [SimTalk]	date, dateTime [SimTalk]
string [SimTalk]	string [SimTalk]

You can use the result of a comparison in nested expressions. You might, for example, concatenate several comparisons with the logical operators **AND**, **OR**, and **NOT**.

Example

```
if index > 0 and index <= DataList.Dim
   print "index in valid range"
end
```

See also

Tolerance for About Equal (~=) Comparison [preferences]

Tolerance For About Equal (~=) Comparison [model settings]

setEpsilon [SimTalk]

The Logical Operators AND and OR

The **logical operators** connect two boolean expressions.

Remarks

The **logical operators** return true under the following conditions:

- The logical operator **AND** only returns *true* if both operands have the value *true*.

AND	false	true
false	false	false
true	false	true

- The logical operator **OR** returns *true* if at least one of the two operands has the value *true*.

OR	false	true
false	false	true
true	true	true

Plant Simulation analyzes a logical combination from left to right. The evaluation ends as soon as the value of the expression is known.

Compare this example:

```
expression1 AND expression2
```

Expression2 is not going to be evaluated if *expression1* returns *false*. Independent of *expression2* the combined result is already *false*. You can use this for programming branching operations and for loop termination conditions.

Check out the following condition:

```
if i <= DataList.dim AND DataList[i] > 0
   // process entry
end
```

Accessing the second expression in the list does not have detrimental effects on the simulation. The first expression (`i <= DataList.Dim`) ensures that the list is only accessed if the index is small enough.

This also is *true* for the operator **OR**. *Plant Simulation* analyzes a logical combination from left to right. As soon as the expression on the left returns the value *true*, *Plant Simulation* stops evaluating. You can use this for programming branching operations and for loop termination conditions.

In our example we have *Plant Simulation* search a list for a value greater than 0.

```
var i := 0
repeat
    i := i + 1
until i > DataList.Dim OR DataList[i] > 0
```

If the list does not contain a value greater than 0, *Plant Simulation* exits the loop when the index is greater by one than the largest index permitted. It then does not access the second part of the expression.

The Logical Operator NOT

The **logical operator NOT** negates a value of data type *boolean*, i.e., it changes *true* to *false* and vice versa.

Examples

```
failure := false
print failure         // outputs false
print NOT failure     // outputs true
print NOT NOT failure // outputs false
```

```
var i,j :real

i:=4.00000001
j:=3.999999

if NOT {i~=j}
```

Assignment Operators

Assignment Operators

SimTalk provides several assignment operators.

- The assignment operator **:=** or **=** respectively
- The reference operator **&**
- **byRef**
- **void**

:= [SimTalk] - assignment operator

The **assignment operator** := assigns a new value to a variable. *Plant Simulation* first evaluates the expression to the right of the operator. If value and variable have the same data type, the value is assigned to the variable.

Remarks

> **Note**
> Instead of := you can also use = as the **assignment operator**.

If the data types are different, the value will have to be converted first. *Plant Simulation* automatically converts *real* into *integer* and *integer* into *real* values. During the conversion of *real* values into *integer*

values *Plant Simulation* deletes the numbers after the decimal point. The same is true for assignments of *date* to *dateTime* and *dateTime* to *date*. The time part will be discarded.

```
integerVar := 2.999 // value is 2, no digits after decimal point
```

Instead, you can also type:

```
integerVar = 2.999 // value is 2, no digits after decimal point
```

You have to explicitly convert all other data types using the respective conversion functions, compare **Manually Converting Data Types**.

Example

```
var r: real; var b: boolean; var s: string; var d: date; var dt: dateTime
r := 1.0 + 4 * (3 - 0.18)                    // 12.28
b := true and (b or true)                    // true
s := "new" + " value"                        // "new value"
dt := str_to_dateTime("2018/1/31 13:45:44")  // date and time
d := dt                                      // date without time, i.e.
midnight
```

Plant Simulation first completely evaluates the right side of an assignment before assigning the value to the variable on the left side. This permits to directly increase the value of a variable.

Example

```
a := a + 3 // integer
```

Suppose that the variable *a*has the value 5, then the right side is formed by the addition of the previous value of *a* and the *integer* value 3, equaling 8. This sum is then assigned to the variable *a*. The previous value will be overwritten!

& [SimTalk] - reference operator

The **reference operator** & accesses attributes or methods of objects.

Remarks

Be aware of the following before you access attributes or methods of objects of type **Variable** or **Method**, or **user-defined attributes** of data type **object** or **json**:

- When accessing a global **Variable**, *Plant Simulation* returns its contents, and therefore you would send the attribute name or method name to the contents and not to the *Variable* object itself.
- When accessing a **Method**, *Plant Simulation* starts executing the *Method* and therefore you would send the attribute name or method name to the result and not to the *Method* object itself.
- When accessing a **user-defined attribute** of data type **object** or **json**, *Plant Simulation* accesses the value of the *user-defined attribute* and not the *user-defined attribute* itself.

The **reference operator** & accesses attributes or methods of one of the objects listed above. The *reference operator* prevents accessing the contents of the *Variable* or calling the *Method* and returns the reference to the object itself. You can then specify this reference or the path in lists or tables and specify them as parameter into other methods.

For object paths you have to insert the **reference operator** & directly in front of the name of the *Method* or the *Variable*, compare the example below.

```
print &MyVariable.DataType
print .UserObjects.&MyVariable.DataType
var obj: object
obj := &MyMethod
obj := &MyVariable
```

```
MyStation.&ObjAttr.InitValue := Buffer     // assigns the initial value of
the user-defined attribute of data type object
MyStation.&JsonAttr.InheritValue := false  // turns off inheritance of the
user-defined attribute of data type JSON
```

If you have a reference to a *global Variable* and if you would like to read the value of the *Variable*, you have to apply the attribute **Value** to the reference.

If you have a reference to a *Method* and if you would like to call the *Method*, you have to call the method **execute** for the reference.

```
MyVariable := .MaterialFlow.Station  // variable of data type object
var obj := &MyVariable
print obj.Name                       // returns MyVariable
print obj.Value.Name                 // returns Station
```

```
obj := &MyMethod                        // reference
obj.execute                             // call
obj := &methodWithArgument
obj.execute(1.0, true,"parameter")   // call with parameter
```

Note

If you are using a *user-defined attribute* of data type *method*, and if you want to access the *attributes* and *methods* of that *user-defined attribute*, then you may not use the **reference operator**.

```
Station.method                 // executes the user-defined method
Station.method.executeIn(60)  // schedules the user-defined method to be
called in 60 seconds
```

See also

& [SimTalk] - Variable

byref [SimTalk] - reference operator

The **byref** operator passes parameters as a reference.

Remarks

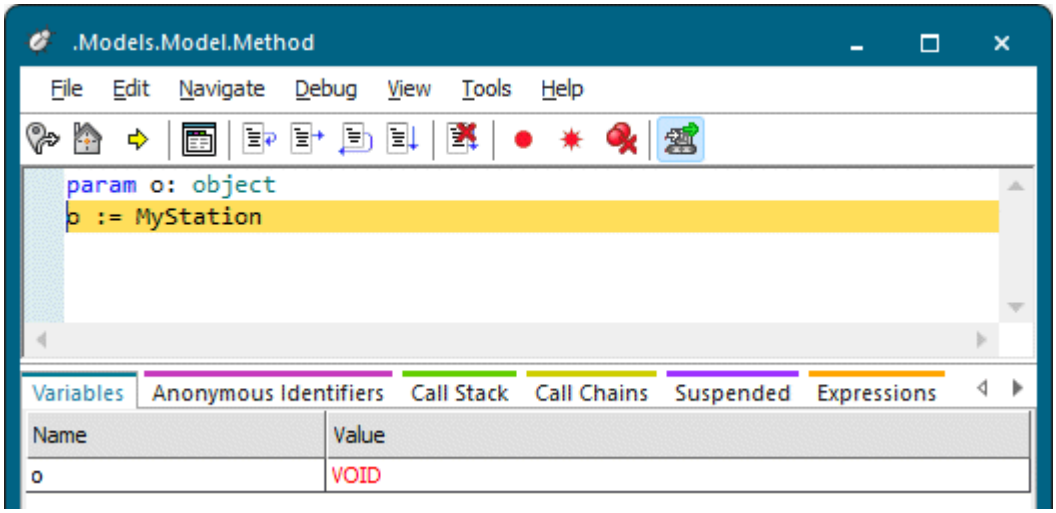
Passing parameters as a reference means that *Plant Simulation* does not copy the value into the parameter, but that the called *Method* directly accesses the called *local variable*. The keyword `byref` can return more than one result to the calling *Method*. You can only specify *local variables*. The data type of the passed *local variable* has to be the exact same data type as the data type of the formal parameter, i.e., *Plant Simulation* does not convert the data types of referenced parameters.

Example

```
param byref a,b : real              // declares method1
a := a + 1
b := b + 1
                                    // declares method2
var x, y : real
print x, " ", y                     // 0 0
method1(x, y)
print x, " ", y                     // 1 1
method1(x, x)
print x, " ", y                     // 3 1
```

void [SimTalk] - variable

Plant Simulation assigns the value *void* to a *local variable* if a *local variable* of data type *object* points to a *Plant Simulation* object that does not exist.



Compatibility of Data Types

Compatibility of Data Types

You can only use mathematical and relational operators with certain data types. The result of the operation may even have a totally different data type.

Remarks

The following tables list the combinations allowed and the data type of the result. If you are, for example, looking for the operation a op b, search the data type a in the row and the data type b in the column. The entry in the table contains the data type of the result.

The data types *time*, *length*, *weight*, *speed* and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real*, or *integer* to a variable of data type *length*.

Compatibility When Assigning a Data Type

Compatibility When Adding Values

Compatibility When Subtracting Values

Compatibility When Multiplying Values

Compatibility When Dividing Values

Compatibility of the Data Type string

Compatibility When Assigning a Data Type

When assigning a value to a variable or an attribute, *Plant Simulation* retains the data type of the variable or attribute. An assignment is only allowed if the data types match, or if the value to be assigned and the variable or attribute both are numerical, except if they have two differing physical units. You can assign a value of data type *length* to an *integer* variable or *real* variable but not to a *speed* variable.

Remarks

Values are only changed for the data type *integer* as the digits after the decimal point are cut off. Allocating `2.5`, for example, returns an *integer* variable with the value 2.

Read the table as follows: The columns contain the data type of the value to be assigned. The rows contain the data type of the Variable or of the attribute to which the value is going to be assigned.

:=	integer	real	length	weight	speed	time	date	datetime	string	object
integer	integer	integer	integer	integer	integer	integer	—	—	—	—
real	real	real	real	real	real	real	—	—	—	—
length	length	length	length	—	—	—	—	—	—	—
weight	weight	weight	—	weight	—	—	—	—	—	—
speed	speed	speed	—	—	speed	—	—	—	—	—
time	time	time	—	—	—	time	—	—	—	—
date	—	—	—	—	—	—	date	date	—	—
datetime	—	—	—	—	—	—	datetime	datetime	—	—
string	—	—	—	—	—	—	—	—	**string**	**string**
object	—	—	—	—	—	—	—	—	**object**	**object**

Compatibility When Adding Values

When adding values to each other, *Plant Simulation* considers the data types *integer* and *real* to be without unit. If the data types of the operands are incompatible, the result has the data type *real*.

Remarks

Read the table as follows: The rows contain the data type of the left operand. The columns contain the data types of the right operand.

+	integer	real	length	weight	speed	time	date	datetime	string
integer	integer	real	length	weight	speed	time	date	datetime	—
real	real	real	length	weight	speed	time	datetime	datetime	—
length	length	length	length	—	—	—	—	—	—
weight	weight	weight	real	weight	—	—	—	—	—
speed	speed	speed	—	—	speed	—	—	—	—
time	time	time	—	—	—	time	datetime	datetime	—
date	date	datetime	—	—	—	datetime	—	—	—
datetime	datetime	datetime	—	—	—	datetime	—	—	—
string	string	—	—	—	—	—	—	—	string

Compatibility When Subtracting Values

When subtracting values from each other, *Plant Simulation* considers the data types *integer* and *real* to be without unit. If the data types of the operands are incompatible, the result has the data type *real*.

Remarks

Read the table as follows: The rows contain the data type of the left operand. The columns contain the data types of the right operand.

-	integer	real	length	weight	speed	time	date	datetime
integer	integer	real	length	weight	speed	time	—	—
real	real	real	length	weight	speed	time	—	—
length	length	length	length	—	—	—	—	—
weight	weight	weight	—	weight	—	—	—	—
speed	speed	speed	—	—	speed	—	—	—
time	time	time	—	—	—	time	—	—
date	date	datetime	—	—	—	datetime	integer (days)	time
datetime	datetime	datetime	—	—	—	datetime	time	time

Compatibility When Multiplying Values

When multiplying values with each other, *Plant Simulation* considers the data types *integer* and *real* to be without unit. If the data types of the operands are incompatible, the result has the data type *real*.

Remarks

Read the table as follows: The rows contain the data type of the left operand. The columns contain the data types of the right operand.

*	integer	real	length	weight	speed	time
integer	integer	real	length	weight	speed	time
real	real	real	length	weight	speed	time
length	length	length	real*	real*	real*	real*
weight	weight	weight	real*	real*	real*	real*
speed	speed	speed	real*	real*	real*	**length**
time	time	time	real*	real*	**length**	real*

*: The result of the multiplication preserves the correct physical unit.

Compatibility When Dividing Values

When dividing values by each other, *Plant Simulation* considers the data types *integer* and *real* to be without unit. If the data types of the operands are incompatible, the result has the data type *real*.

Remarks

Read the table as follows: The rows contain the data type of the left operand. The columns contain the data types of the right operand.

/	integer	real	length	weight	speed	time
integer	integer	real	real*	real*	real*	real*
real	real	real	real*	real*	real*	real*
length	length	length	real	real*	**time**	**speed**
weight	weight	weight	real*	real	real*	real*
speed	speed	speed	real*	real*	real	**acceleration**
time	time	time	real*	real*	real*	real

*: The result of the division preserves the correct physical unit.

Compatibility of the Data Type string

The data type *string* only recognizes the addition operator (+). *Plant Simulation* combines the two strings to a new string, meaning that it appends the second string to the first string.

Remarks

We type in this instruction:

```
print "My short string " + "plus my long string with many words."
```

The output in the *Console* looks like this:

As an exception you can also add an *integer* value to a *string*. *Plant Simulation* converts the *integer* value to a *string* and appends it.

```
param obj : object -> string
return obj.Name + obj.Label + obj.XPos
```